

Foundations of Spatial AI

Lecture 03 — Single-View Depth II

Detailed lecture notes

Notes on the slides of L. Di Giammarino and E. Giacomini (Sapienza University of Rome)

Lecture of September 30, 2026

Abstract

Part I of the course obtained depth from a sensor (a laser, paid for in time) or from a network fitted to sensor labels (a prior, paid for in labels). Part II, covered here, asks how far a prior can be pushed when labels are scarce or absent: (i) *self-supervision* from stereo pairs, (ii) *diffusion* as a generative prior, (iii) *Marigold*, a depth estimator obtained by fine-tuning an image diffusion model, (iv) how affine errors distort the 3D reconstruction, and (v) *Marigold-DC*, which steers the sampling loop with a few metric measurements. Section titles and their slide ranges follow the slide deck; slide numbers refer to the printed page numbers of the PDF.

About these notes. Only the slide deck was provided (no companion book chapter). Formal statements are therefore taken from the slides themselves; where a proof or a derivation is not on the slides I supply it, and derivations that go beyond the slides are collected in Appendix ???. Blue *Intuition* boxes explain the ideas in plain words. Photographs and rendered result images of the slides are replaced by dashed placeholders that say which slide to copy from; all conceptual diagrams are redrawn in TikZ.

Contents

1 Part I in One Slide (*slides 1–2*)

Part I obtained depth in two ways and closed *owing two debts*:

- **Time.** A laser measures range directly: metric and exact where it lands, but sparse, and it produces the labels everything else trains on.
- **A prior.** A network fitted to those labels is dense and cheap, but it *predicts* rather than recovers depth. The prior was fitted to labels that a sensor had to measure.

Mixing labels of different origin moved the output down a table, at a cost of free parameters.

Definition 1.1 (Output types and their free parameters). A depth-like output is defined only up to a transformation, and each transformation leaves free parameters that something else must later fix.

output	free parameters	aligned by
metric depth	0	nothing
scale-invariant depth	1	one scale s
affine-invariant disparity or depth	2	scale and shift (s, t)
ordinal	any monotone map	nothing

Intuition

Each free parameter is a *number a sensor must still supply*. If a network says “this pixel is twice as far as that one”, it has told you shape but not meters. Example: the truth is $D = 2.0 \hat{D} + 0.3$ m. A metric network outputs D itself; an affine-invariant one outputs $\hat{D}$ and you need the pair $(2.0, 0.3)$ to recover meters. The whole lecture is about who supplies those numbers: the baseline of a stereo rig, a large generative prior, or a handful of LiDAR returns.

2 Supervision without Ground Truth (*slides 3–6*)

This is section 3.9 of the lecture. Its thesis: *a stereo pair supplies its own supervision, provided the warp connecting the two images is differentiable*.

2.1 View Synthesis and the Backward Warp (*slides 4*)

Training uses stereo pairs $\{(\mathbf{I}_L, \mathbf{I}_R)\}$ only; there is no depth anywhere in the training set. The objective is to reproduce one image of the pair from the other. Depth is the only internal quantity that makes this task solvable, so if the network solves it, depth is what it must have learned. This holds only where a pixel is **textured** and visible in **both** views; elsewhere the objective says nothing and a hand-written prior fills the gap.

Definition 2.1 (Backward warp). Given a disparity map $d(u, v)$ defined on the *target* image, the backward warp synthesizes the target from the source by

$$\tilde{\mathbf{I}}_L(u, v) = \mathbf{I}_R(u - d(u, v), v). \quad (2.1)$$

Every *target* pixel is mapped to exactly one *source* location.

Proposition 2.2 (Why the warp must be backward). *Forward mapping (pushing each source pixel to a target location) lets several sources land on one target pixel and leaves other target pixels empty: collisions and holes. Backward mapping assigns each target pixel exactly one value, which makes the warp a differentiable function of d rather than a scatter operation.*

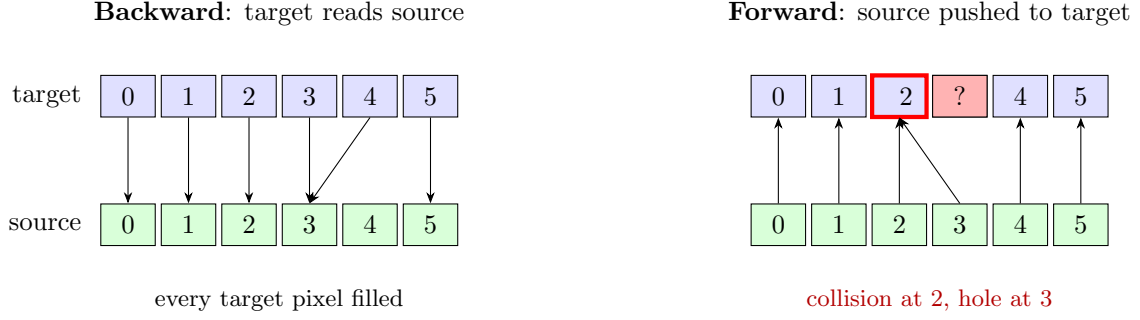


Figure 1: Original 1D illustration of backward versus forward mapping (slide 4 shows the same effect on small coloured grids).

Intuition

Think of filling in a form versus mailing letters. Backward warping is the form: each target pixel asks “*where in the other image should I look?*” and gets exactly one answer. Forward warping is the post office: every source pixel is mailed to an address; some mailboxes get two letters (collision), others none (hole), and “two letters or none” has no useful gradient. Note the sign convention: slide 4 writes $u - d$ for the left image as target, slide 5 writes $+d$ for a target/source pair set up the other way round; only the consistent use matters.

2.2 Image Reconstruction as Supervision (*slides 5*)

The pipeline of slide 5 has three steps: (1) a deep CNN predicts an inverse depth from the left image; (2) the right image is warped with it into a synthetic left image; (3) the reconstruction error against the real left image is the loss.

Definition 2.3 (Disparity and photometric loss). Under a rectified stereo rig with focal length f and baseline b , the predicted inverse depth is the disparity

$$d(\mathbf{p}) = \frac{fb}{z(\mathbf{p})}. \quad (2.2)$$

The self-supervised loss compares the warped and real image,

$$\mathcal{L}_{\text{photo}} = \sum_{\mathbf{p}} \|\tilde{\mathbf{I}}(\mathbf{p}) - \mathbf{I}_L(\mathbf{p})\|, \quad \tilde{\mathbf{I}}(\mathbf{p}) = \mathbf{I}_R(\mathbf{p} + d(\mathbf{p})). \quad (2.3)$$

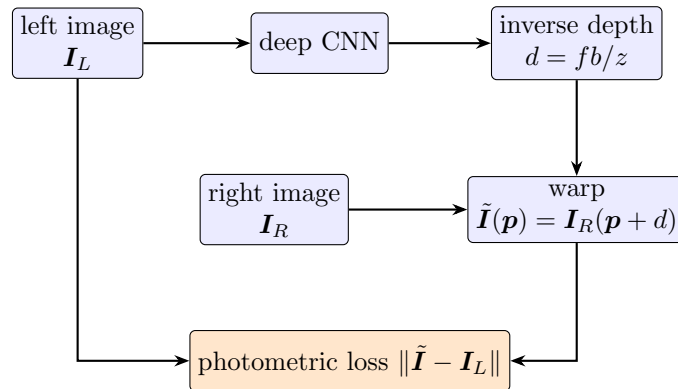


Figure 2: Self-supervised stereo training (redrawn from slide 5). No depth label appears anywhere.

[PHOTO PLACEHOLDER]

Insert the image from **slide 5**:

the KITTI street scene: left image, predicted inverse-depth map, warped image and reconstruction-error map (steps 1–3, with the red marker point)

Intuition

Depth is the *only* unknown the network can vary to make the warped image match the real one, so a network that solves the task has learned depth. Worked numbers (illustrative): with $f = 700$ px and $b = 0.5$ m, a car at $z = 10$ m has $d = 35$ px, whereas at $z = 70$ m it has only $d = 5$ px. Disparity is *inverse* depth: distant objects move the image little, so the training signal is weak at range.

2.3 Where the Loss Is Blind (*slides 6*)

Proposition 2.4 (Blind spots of the photometric loss). (a) Textureless regions. Writing the residual as $r = \mathbf{I}_R(u - d) - \mathbf{I}_L(u)$ and the squared loss as $\frac{1}{2}r^2$,

$$\frac{\partial \mathcal{L}}{\partial d} = -r \cdot \mathbf{I}'_R(u - d), \quad (2.4)$$

which vanishes wherever $\mathbf{I}(u_0 + 1) \approx \mathbf{I}(u_0)$ (flat image, zero derivative): such pixels receive no training signal at all.

(b) Occlusions. A pixel visible to one camera only has no match in the other view, so the residual there is meaningless.

Both gaps are filled by hand-written priors: a smoothness term, left-right consistency, a minimum over the two reprojections, and masking of pixels that do not move between frames.

Proposition 2.5 (The baseline decides what the trained model is worth). Trained on a stereo pair, the model inherits the known baseline b and is metric for that rig. Trained on video, the baseline is unknown, so the output is shape up to one global scale and every image needs its own alignment.

Intuition

The photometric loss is a detective that only works with clues. Flat white wall: no clue, no gradient, so the smoothness prior guesses. Something hidden from one eye: no partner to compare with. The baseline is the ruler: with stereo the ruler length b is known, so meters come free; with a monocular video the ruler length is not known, so you get a beautiful shape in unknown units (one free parameter, as in the table of Section 1).

3 Diffusion, From Scratch (*slides 7–22*)

This is section 3.10. Its thesis: *a model trained to remove noise is a prior: it points toward the data from anywhere in the space, and it can be sampled, conditioned or steered.* (Adapted on the slides from MIT 6.S183, Lecture 1, by D. Pfaffner.)

3.1 Supervised Learning (*slides 8*)

Supervised learning fits **one output per input**. At a depth edge that output is the average of the near surface and the far one: a distance that belongs to neither.

Proposition 3.1 (MSE regression returns the conditional mean). *Let Y be the target and $\mathbf{x}$ the input. Among all functions f , the minimizer of $\mathbb{E}\|f(\mathbf{x}) - Y\|^2$ is $f^*(\mathbf{x}) = \mathbb{E}[Y | \mathbf{x}]$.*

Proof. For fixed $\mathbf{x}$ with $m = \mathbb{E}[Y | \mathbf{x}]$, $\mathbb{E}[\|f - Y\|^2 | \mathbf{x}] = \|f - m\|^2 + \mathbb{E}[\|Y - m\|^2 | \mathbf{x}]$; the second term does not depend on f , and the first is minimized by $f = m$. $\square$

Intuition

At a depth edge half the training examples say 2 m (near surface) and half say 10 m (far surface). The best single answer under squared error is 6 m, which is the depth of *nothing*. That is the blurred, flying-pixel edge of regression networks.

3.2 Generative Learning (*slides 9*)

A generative model fits a **distribution** over outputs, not one value. “The band is what a diffusion model represents.”

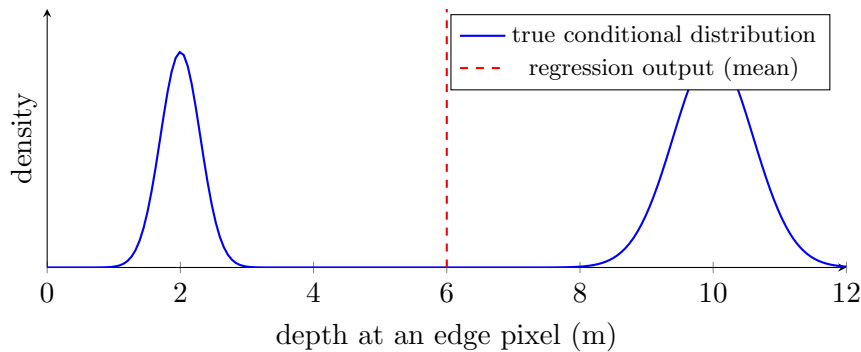
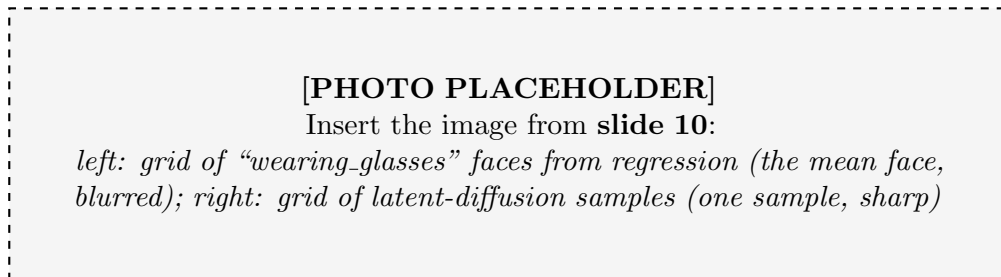


Figure 3: Original toy figure: a bimodal conditional distribution at a depth edge. The regressor answers the mean, which lies in a gap where the density is zero; a generative model can return a sample from one of the two modes.

3.3 Many Outputs for One Input – Text to Image (*slides 10*)

One conditioning input has many valid completions. A regressor averages them; a diffusion model picks one.



Intuition

Blur is not a training failure. It is the correct answer to the wrong question: the conditional mean of a set of sharp images is not a sharp image. The same averaging is what flattens a

depth edge.

3.4 The Forward and Reverse Process (*slides 11*)

The **forward process** adds noise to data one step at a time until nothing of the data is left. It is fixed and has no parameters. It is inspired by Brownian motion (many small independent displacements), and after enough steps the sample is indistinguishable from Gaussian noise. A diffusion model learns the **reverse process**, which recovers data from noise; that reverse process *is the prior*.

Definition 3.2 (Forward and reverse processes). The forward chain $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ maps $\mathbf{x}_0$ (data) to $\mathbf{x}_T$ (noise) and is fixed; the model $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ learns the way back.

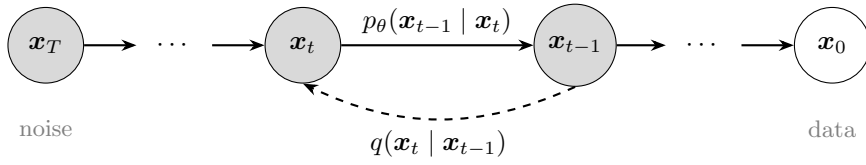


Figure 4: Forward (dashed, fixed) and reverse (solid, learned) chains, redrawn from slide 11.

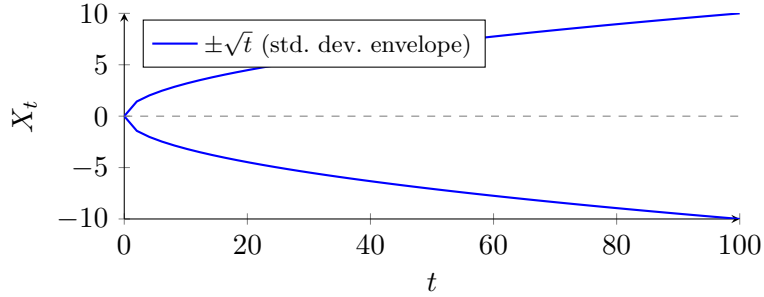


Figure 5: Original figure: the spread of a random walk grows like $\sqrt{t}$, the mechanism that buries the data in noise.

[PHOTO PLACEHOLDER]
 Insert the image from **slide 11**:
Brownian-motion simulated paths (coloured bundle of trajectories) with the histogram of X_T on the right, and the two small face images (noisy, clean) below the chain

3.5 Training by Noise Prediction (*slides 12*)

A denoising diffusion model estimates the noise vector ϵ from a noise level $\sigma > 0$ and a noisy input $\mathbf{x}_\sigma \approx \mathbf{x}_0 + \sigma\epsilon$, where $\mathbf{x}_0$ lies on the data manifold $\mathcal{K} \subset \mathbb{R}^n$.

Definition 3.3 (Denoiser and training loss). A denoiser $\epsilon_\theta : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$ is learned by minimizing

$$L_{\text{simple}}(\theta) := \mathbb{E}_{\mathbf{x}_0, \sigma, \epsilon} \left[\left\| \epsilon_\theta(\mathbf{x}_0 + \sigma\epsilon, \sigma) - \epsilon \right\|^2 \right], \quad (3.1)$$

with $\mathbf{x}_0$ sampled from the training data, σ from a training noise schedule, and $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Intuition

The recipe is three lines: take a clean example, add noise of a random strength you remember, and ask the network “what noise did I add?”. Subtracting the guessed noise gives the guessed clean point, $\hat{x}_0 = x_\sigma - \sigma \epsilon_\theta$. No labels beyond the data itself are needed.

3.6 The Spiral Toy Model (*slides 13*)

Everything that follows is shown on a 2D data set, where the prior is visible: *it is where the points are*. The data are points of a 2D spiral x_0 . The network is a fully connected MLP fed with $x_0 + \sigma \epsilon$ and a 2D noise-level embedding of σ , and it predicts the noise.

```
def get_sigma_embeds(sigma):
    sigma = sigma.unsqueeze(1)
    return torch.cat([torch.sin(torch.log(sigma)/2),
                      torch.cos(torch.log(sigma)/2)], dim=1)

class TimeInputMLP(nn.Module):
    def __init__(self, dim, hidden_dims):
        super().__init__()
        layers = []
        for in_dim, out_dim in pairwise((dim + 2,) + hidden_dims):
            layers.extend([nn.Linear(in_dim, out_dim), nn.GELU()])
        layers.append(nn.Linear(hidden_dims[-1], dim))
        self.net = nn.Sequential(*layers)
        self.input_dims = (dim,)

    def forward(self, x, sigma):
        sigma_embeds = get_sigma_embeds(sigma)
        nn_input = torch.cat([x, sigma_embeds], dim=1)
        return self.net(nn_input)

model = TimeInputMLP(dim=2, hidden_dims=(16,128,128,128,128,16))
```

Intuition

The extra two inputs (sin, cos) of $\log \sigma/2$ tell the network *how noisy* its input is. This matters because the right answer depends on σ : at small σ the noise is a small nudge off the nearest data point, at large σ the input is almost pure noise. Note that the first layer has width $\dim + 2 = 4$: two coordinates plus the two embedding numbers.

3.7 Training the Toy Model (*slides 14*)

```
def training_loop(loader, model, schedule, epochs=10000):
    optimizer = torch.optim.Adam(model.parameters())
    for _ in range(epochs):
        for x0 in loader:
            optimizer.zero_grad()
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = model(x0 + sigma * eps, sigma)
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()
```

The learned denoiser can be drawn as a **vector field** by plotting $x - \sigma \epsilon_\theta(x, \sigma)$. At small noise the arrows point at the nearest data. At large noise they point at the middle of the whole data set. The slide’s training curve falls from about 0.70 to about 0.41 over 15 000 epochs.

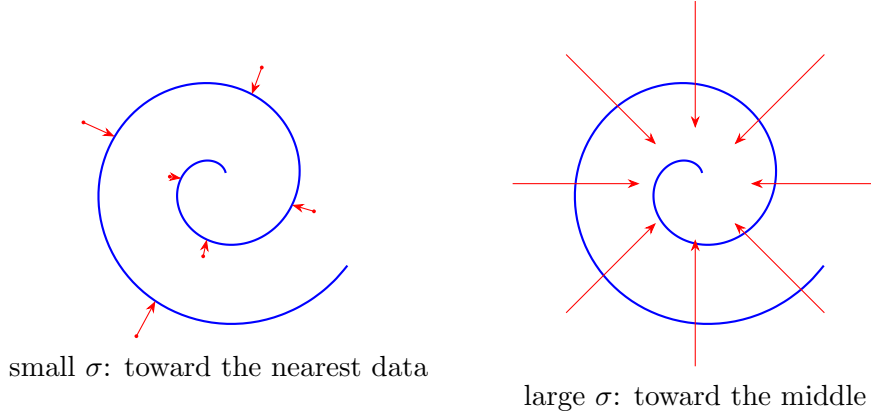


Figure 6: Original sketch of the denoiser field on the spiral (compare slide 14).

Remark 3.4 (Why the loss does not reach zero). The curve plateaus near 0.41, not 0. This is expected: even the best possible denoiser cannot recover ϵ exactly, because many $(\mathbf{x}_0, \epsilon)$ pairs produce the same $\mathbf{x}_\sigma$; the floor equals the average posterior variance of ϵ (Appendix ??, item ??).

3.8 The Ideal Denoiser (*slides 15*)

The **ideal denoiser** minimizes the training loss; it is a function of the data set $\mathcal{K}$ and the noise level, and nothing else:

$$\epsilon^* := \arg \min_{\epsilon_\theta} \mathbb{E}_{\mathbf{x}_0, \sigma, \epsilon} \|\epsilon_\theta(\mathbf{x}_0 + \sigma\epsilon, \sigma) - \epsilon\|^2. \quad (3.2)$$

Theorem 3.5 (Ideal denoiser for a finite data set). *For a finite $\mathcal{K}$ (data uniformly distributed on $\mathcal{K}$, Gaussian noise),*

$$\epsilon^*(\mathbf{x}_\sigma, \sigma) = \frac{\sum_{\mathbf{x}_0 \in \mathcal{K}} (\mathbf{x}_\sigma - \mathbf{x}_0) \exp(-\|\mathbf{x}_\sigma - \mathbf{x}_0\|^2 / 2\sigma^2)}{\sigma \sum_{\mathbf{x}_0 \in \mathcal{K}} \exp(-\|\mathbf{x}_\sigma - \mathbf{x}_0\|^2 / 2\sigma^2)}. \quad (3.3)$$

It is a weighted average of all data points: nearer points count more.

Proof. By the conditional-mean proposition, $\epsilon^*(\mathbf{x}_\sigma, \sigma) = \mathbb{E}[\epsilon \mid \mathbf{x}_\sigma]$ with $\epsilon = (\mathbf{x}_\sigma - \mathbf{x}_0)/\sigma$. By Bayes, $p(\mathbf{x}_0 \mid \mathbf{x}_\sigma) \propto \exp(-\|\mathbf{x}_\sigma - \mathbf{x}_0\|^2 / 2\sigma^2)$ on $\mathcal{K}$. Taking the expectation of $(\mathbf{x}_\sigma - \mathbf{x}_0)/\sigma$ under these weights gives the formula. $\square$

Intuition

Worked 1D example with $\mathcal{K} = \{-1, +1\}$, $\mathbf{x}_\sigma = 0.2$. For $\sigma = 0.5$ the weights are $e^{-1.44/0.5} \approx 0.056$ (for -1) and $e^{-0.64/0.5} \approx 0.278$ (for $+1$), giving $\epsilon^* \approx -0.93$ and clean estimate $\hat{\mathbf{x}}_0 = 0.2 - 0.5 \cdot (-0.93) \approx 0.66$: pulled toward the nearer point $+1$. For a large $\sigma = 5$ the two weights are almost equal (0.972 vs 0.987), $\epsilon^* \approx 0.04$ and $\hat{\mathbf{x}}_0 \approx 0.01$: the arrow points at the middle of the data set, exactly as in the slide's three panels ($\sigma = 0.01, 0.1, 0.5$).

3.9 The Smoothed Distance Function (*slides 16*)

The distance to a set, $\text{dist}_{\mathcal{K}}(\mathbf{x}) := \inf\{\|\mathbf{x} - \mathbf{x}_0\| : \mathbf{x}_0 \in \mathcal{K}\}$, is not differentiable everywhere, so its gradient is hard to learn. Smoothing it by σ fixes that.

Definition 3.6 (Smoothed distance).

$$D_\sigma(\mathbf{x}) := -\sigma^2 \log \left(\sum_{\mathbf{x}_0 \in \mathcal{K}} \exp(-\|\mathbf{x}_0 - \mathbf{x}\|^2 / 2\sigma^2) \right). \quad (3.4)$$

(The slide denotes the smoothed squared distance by $\text{dist}_{\mathcal{K}}^2(\mathbf{x}, \sigma)$ and writes the identity below with a factor $\frac{1}{2}$; with the definition as printed, $\frac{1}{2}\text{dist}_{\mathcal{K}}^2(\mathbf{x}, \sigma)$ corresponds to D_σ .)

Proposition 3.7 (Denoiser = gradient of the smoothed distance). $\frac{1}{2}\nabla_{\mathbf{x}} \text{dist}_{\mathcal{K}}^2(\mathbf{x}, \sigma) = \nabla_{\mathbf{x}} D_\sigma(\mathbf{x}) = \sigma \boldsymbol{\epsilon}^*(\mathbf{x}, \sigma)$, and $\min_{\mathbf{x}_0} \frac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|^2 - \sigma^2 \log |\mathcal{K}| \leq D_\sigma(\mathbf{x}) \leq \min_{\mathbf{x}_0} \frac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|^2$.

Proof. Differentiating, $\nabla D_\sigma = -\sigma^2 \cdot \frac{\sum e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2 / 2\sigma^2} (-(\mathbf{x} - \mathbf{x}_0) / \sigma^2)}{\sum e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2 / 2\sigma^2}} = \frac{\sum (\mathbf{x} - \mathbf{x}_0) e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2 / 2\sigma^2}}{\sum e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2 / 2\sigma^2}} = \sigma \boldsymbol{\epsilon}^*$ by Theorem ?? . The bounds follow from $e^{-a_{\min}/\sigma^2} \leq \sum_i e^{-a_i/\sigma^2} \leq |\mathcal{K}| e^{-a_{\min}/\sigma^2}$ with $a_i = \frac{1}{2}\|\mathbf{x} - \mathbf{x}_{0,i}\|^2$. $\square$

[PHOTO PLACEHOLDER]

Insert the image from **slide 16**:

the two coloured distance-field images of the cat silhouette (“On Surface” and “In Surface” with the dotted arrows), with the colour bar 0–0.6

Intuition

min has kinks where two data points are equally near; a *soft*-min has none, and its slope is the vector from the weighted mean of the data to $\mathbf{x}$. So the denoiser is a compass that is defined *everywhere* and always points toward the data: that is what makes it a prior. Equivalently $\hat{\mathbf{x}}_0 = \mathbf{x}_\sigma - \sigma \boldsymbol{\epsilon}^*$ is simply the weighted mean of the data points.

3.10 Sampling From a Diffusion Model – Why iterative? (slides 17)

Given a noisy $\mathbf{x}_\sigma$ and its noise level, the denoiser estimates the clean point directly,

$$\hat{\mathbf{x}}_0(\mathbf{x}_\sigma, \sigma) := \mathbf{x}_\sigma - \sigma \boldsymbol{\epsilon}_\theta(\mathbf{x}_\sigma, \sigma). \quad (3.5)$$

A sampler walks a decreasing sequence of noise levels, taking one such estimate at each step. **DDIM** is the update

$$\mathbf{x}_{t-1} = \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, \sigma_t). \quad (3.6)$$

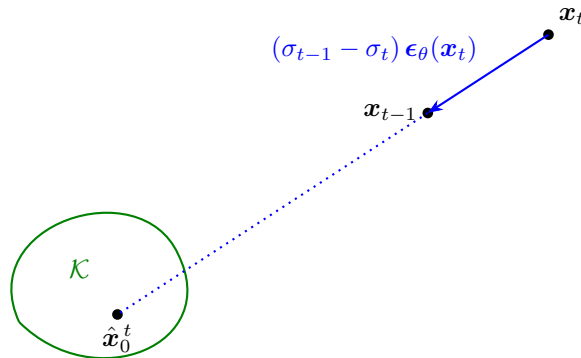


Figure 7: DDIM step (redrawn from slide 17): a straight line toward the current estimate $\hat{\mathbf{x}}_0^t$ of the data.

Intuition

One step goes only *part* of the way to the current best guess, because the guess is poor while the input is very noisy; after moving, you re-evaluate the compass. The step is a straight line toward the estimate. This is the equation that depth completion reaches inside and steers (Section ??). Algebraically the step re-noises the clean estimate to the lower level: $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_0^t + \sigma_{t-1}\epsilon_\theta$ (Appendix ??).

3.11 The DDIM Sampler (*slides 18*)

Algorithm 3.8 (DDIM sampler). **for** $t = N, \dots, 1$: $\mathbf{x}_{t-1} \leftarrow \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \epsilon_\theta(\mathbf{x}_t, \sigma_t)$;
return $\mathbf{x}_0$.
 Start from $\mathbf{x}_N = \sigma_N \cdot \mathcal{N}(\mathbf{0}, \mathbf{I})$.

```
@torch.no_grad()
def samples_ddim(model, sigmas, batchsize=1):
    xt = model.rand_input(batchsize) * sigmas[0]
    for sig, sig_prev in pairwise(sigmas):
        xt = xt - (sig - sig_prev) * model.predict_eps(xt, sig)
    return xt
```

On the toy model: *fifty steps, no retraining, and the trajectory follows the field.*

3.12 Probabilistic Sampling (*slides 19*)

DDIM is deterministic: the same starting noise always gives the same sample. Adding a noise term back makes the sampler stochastic:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \epsilon_\theta(\mathbf{x}_t, \sigma_t) + \mu \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3.7)$$

(The slide writes $\sigma_{t'}$ for the next, lower level.) With $\mu = 0$ this is DDIM again; as μ grows the trajectories spread, and the same input gives different samples. The slide shows $\mu = 0.001, 0.01, 0.1$.

Intuition

μ is a “temperature”: $\mu = 0$ rolls a ball down a fixed groove, $\mu > 0$ shakes the table on the way. This is the reason different seeds give different depth maps in Marigold (Section ??).

3.13 Three Uses of a Prior (*slides 20*)

A prior that exists as a **separate object** can be used in three ways, each a later section of the lecture.

use	meaning	section
Sample	draw from the distribution the model represents	3.10
Condition	train it with a side input $\mathbf{c}$, so it samples from $p(\mathbf{x} \mathbf{c})$	3.11
Guide	modify the state inside the sampling loop at test time, with no retraining	3.13

The third is the one that turns a depth prior into a metric depth map.

[PHOTO PLACEHOLDER]

Insert the image from **slide 20**:

the three panels: sampler comparison plots (Algorithm 1 vs Algorithm 2), the generated landscape (conditioning), and the super-resolution / inpainting / colorization examples (guidance)

3.14 The Autoencoder and Latent Diffusion (*slides 21*)

Images are big, so an autoencoder compresses them first: an encoder $\mathcal{E}$ maps $\mathbf{x}$ to a latent $\mathbf{z}$, and a decoder $\mathcal{D}$ maps back, $\mathbf{z} = \mathcal{E}(\mathbf{x})$, $\tilde{\mathbf{x}} = \mathcal{D}(\mathbf{z})$. The autoencoder is trained **first**, then **frozen**; the diffusion model never sees pixels and you decode once, at the end. Diffusion runs on $\mathbf{z}$ with the same loss:

$$L = \mathbb{E}_{\sigma, \mathbf{z}_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\mathbf{z}_0 + \sigma \epsilon, \sigma) \right\|^2 \right]. \quad (3.8)$$

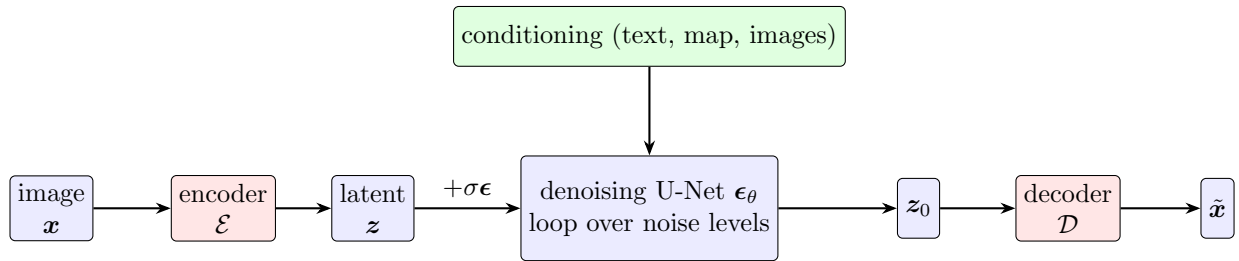


Figure 8: Latent diffusion in outline (original, inspired by slide 21). Pink boxes are frozen autoencoder parts.

Intuition

Worked numbers: $768 \times 768 \times 3 \rightarrow 96 \times 96 \times 4$, i.e. 1 769 472 numbers become 36 864: eight times smaller per side, four channels, **48 times fewer numbers**. Each of the 50 denoising steps is therefore 48 times cheaper than it would be in pixel space.

3.15 Conditioning and Guidance (*slides 22*)

Conditioning is a side input the network was *trained with*; the model responds to it because it was trained to. **Guidance** modifies the sampling trajectory *at test time*, using a criterion the network was never trained on. Sampling is a loop, and its state can be modified between steps.

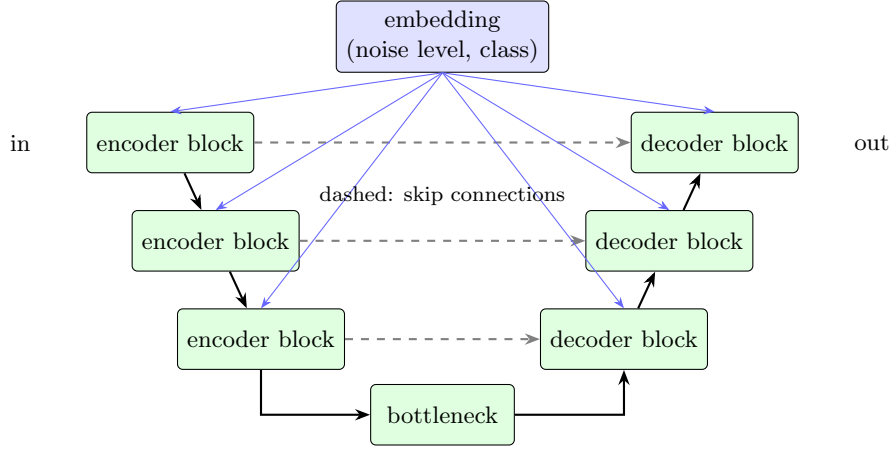


Figure 9: Simplified U-Net redrawn from slide 22. Each block is GroupNorm–SiLU–Conv3×3 with the embedding injected through a Linear layer (scale and shift); attention is optional.

4 Marigold (*slides 23–31*)

Section 3.11: a depth estimator from an image diffusion model. Stable Diffusion’s autoencoder is kept, only its denoiser is fine-tuned, and the output is a depth map rather than an image.

4.1 The Marigold Method (*slides 24*)

The argument: a monocular estimator’s knowledge of the visual world is **bounded by its training set**, while a text-to-image model trained on billions of captioned photographs has already seen it. Hence: if the cornerstone of monocular depth is a comprehensive representation of the visual world, one should be able to derive a depth estimator from a pretrained image diffusion model.

Definition 4.1 (Marigold). Take a pretrained text-to-image latent diffusion model, **freeze the VAE**, and fine-tune only the denoising U-Net so that what it denoises is a *depth* latent rather than an image latent. Properties from the slide: affine-invariant monocular depth; trained on **synthetic data only**; about 2.5 days on a single consumer GPU.

[PHOTO PLACEHOLDER]

Insert the image from **slide 24**:

the strip of example results: photographs (beads, church, marigolds, dog, sculpture, forest, painting) with their depth maps and back-projected 3D views

4.2 From the Spiral to Marigold (*slides 25*)

Marigold is the spiral model with one extra input; nothing in the algorithm changes.

	toy model	Marigold
starts from	random weights	Stable Diffusion’s weights
a data point	a 2D point	a depth latent
the denoiser	a small MLP	Stable Diffusion’s U-Net
the condition	none	the image latent
trained on	spiral points	74K rendered image–depth pairs
the sampler	DDIM	the same DDIM
a sample is	a point on the spiral	a depth map the RGB allows

The first row is the whole bet: *the visual prior is already in those weights*, and fine-tuning moves what they denoise from images to depth. (Stable Diffusion indexes noise levels by a timestep t and scales its input; both are fixed functions of σ , so the process is the same.)

4.3 The Source of Affine Invariance (*slides 26*)

The normalization defines the output space: the affine-invariant *depth* row of the table in Part I, not the affine-invariant *disparity* row that MiDaS uses.

Definition 4.2 (Percentile normalization). With D_2 and D_{98} the 2nd and 98th percentiles of the depth map D , the training target is

$$\tilde{D} = \left(\frac{D - D_2}{D_{98} - D_2} - 0.5 \right) \times 2, \quad \text{and metric depth is recovered as} \quad \tilde{D}_{\text{metric}} = s\tilde{D} + t. \quad (4.1)$$

Proposition 4.3 (Normalization removes scale and shift). *If $D' = aD + b$ with $a > 0$, then $\tilde{D}' = \tilde{D}$.*

Proof. Percentiles commute with increasing affine maps: $D'_2 = aD_2 + b$, $D'_{98} = aD_{98} + b$. Hence $(D' - D'_2)/(D'_{98} - D'_2) = a(D - D_2)/(a(D_{98} - D_2))$, the same ratio. $\square$

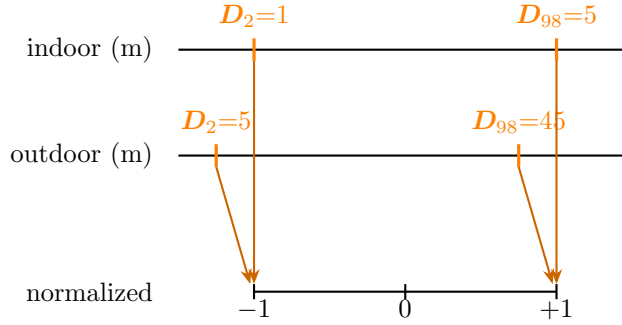


Figure 10: Original figure: an indoor map (few meters) and an outdoor map (tens of meters) land on the same $[-1, +1]$ range; each map is normalized by its *own* percentiles (slide 26). The numbers are illustrative.

Intuition

Every training map is stretched to fill $[-1, +1]$, so a room and a highway look identical to the network in terms of range. **The model never sees meters and never sees a common zero, so it cannot learn them; it is constrained to predict shape.** Example: in a room with $D_2 = 1$ m and $D_{98} = 5$ m, a pixel at 3 m maps to 0; on a street with $D_2 = 5$ m and $D_{98} = 45$ m, a pixel at 25 m also maps to 0.

4.4 Conditioning on the Image (*slides 27*)

The denoiser takes the noisy depth latent $z_\sigma^{(D)}$ and the image latent $z^{(I)}$:

$$\epsilon_\theta(z_\sigma^{(D)}, z^{(I)}, \sigma), \quad z_\sigma = \text{cat}(z_\sigma^{(D)}, z^{(I)}). \quad (4.2)$$

The depth map is replicated to three channels and encoded by the **frozen RGB autoencoder**, whose reconstruction error on depth is negligible. **Concatenation** suits a condition that is spatially aligned with the output, pixel for pixel; cross-attention is for conditions that are not.

Definition 4.4 (Four changes to the network). (1) Concatenate the image latent and the noisy depth latent along the channels; (2) double the U-Net’s input channels; (3) duplicate the first layer’s weights and **halve** them, so activation magnitudes do not inflate; (4) **disable text conditioning** (original Stable Diffusion).

Intuition

Why halve? Let the original first layer be Wz . The new layer sees $[z; z]$ and uses $[W/2, W/2]$, so its output is $Wz/2 + Wz/2 = Wz$: at the start of fine-tuning the pretrained network behaves as before. Without halving, the same input would produce $2Wz$, twice the activations the pretrained weights expect. Since the latent has 4 channels, the new input has 8. Concatenation works here because the depth map and the image are aligned pixel by pixel.

4.5 Training Marigold (*slides 28*)

```
def training_loop(loader, unet, vae, schedule, epochs):
    optimizer = torch.optim.Adam(unet.parameters())
    for _ in range(epochs):
        for image, depth in loader: # pairs
            optimizer.zero_grad()
            # start lock
            x0 = vae.encode(normalize(depth)) # a depth latent
            cond = vae.encode(image) # the condition
            # end lock
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = unet(x0 + sigma*eps, cond, sigma) # sees both
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()
```

The objective is the one from the spiral, unchanged. Only the U-Net trains; the VAE is frozen at both ends. Data: 74K rendered pairs, about 2.5 days on one consumer GPU.

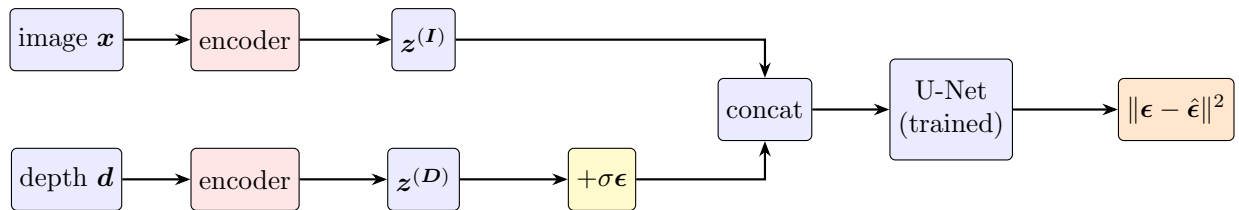


Figure 11: Marigold training (redrawn from slides 27–28). Pink: frozen. The noise ϵ is also fed to the loss.

4.6 Inference (*slides 29*)

Algorithm 4.5 (Marigold inference, one run). (1) Encode the image **once**; (2) initialize the depth latent $z_T^{(D)}$ as pure Gaussian noise; (3) run the denoising loop, concatenating the image latent at **every** step; (4) decode $z_0^{(D)}$ and average the three channels. From the paper: 50 DDIM steps.

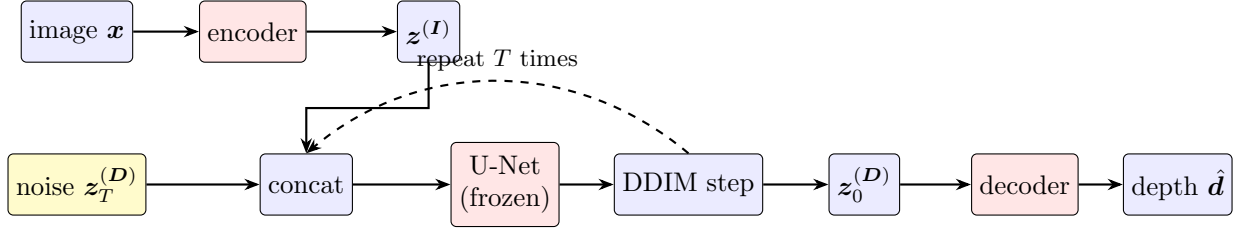


Figure 12: Marigold inference (redrawn from slide 29).

4.7 Ensembling Over Samples (*slides 30*)

Probabilistic sampling is why different seeds give different depth maps; Marigold runs the loop N times and combines the results.

Algorithm 4.6 (Ensembling). (1) Each run is affine-invariant on its own scale, so first fit a scale and a shift per run so that the maps agree with each other (minimize the pairwise distance between the aligned maps, with a penalty that keeps the range on $[0, 1]$); (2) take the pixel-wise **median**. Default $N = 10$; **no ground truth is used**.

Intuition

Ten drafts of the same drawing, each in its own unknown units. First rescale and shift each draft to line up with the others (they only agree up to (s_i, t_i)), then let the median vote at each pixel, which ignores outlier drafts.

4.8 Limitations of Marigold (*slides 31*)

- **No metric scale.**
- **No zero:** not even a common origin between two images of the same room.
- **Slow:** 50 denoising steps $\times$ 10 ensemble members = 500 network evaluations per image.
- **It returns a sample** (non-deterministic).

5 Back to 3D (*slides 32–34*)

Section 3.12: back-projection turns a depth map into points and shows which of the unknown parameters distort the result.

5.1 Effect of Scale and Shift Errors (*slides 33*)

A depth map is a 2.5D surface: one depth per pixel, with nothing behind what is visible.

Definition 5.1 (Back-projection). A pixel with intrinsics-normalized ray $\mathbf{r} = \mathbf{K}^{-1}(u, v, 1)^\top$ and depth z back-projects to $\mathbf{p} = z\mathbf{r}$.

Proposition 5.2 (Affine depth error). If $\hat{z} = sz + t$, the reconstructed point is $\hat{\mathbf{p}} = (s + t/z)\mathbf{p}$. In particular: the **scale** s multiplies the whole cloud (a similarity, so shape is preserved); the **shift** t drags every point a constant depth along its own ray, a different displacement in every direction (the shape can distort).

Proof. $\hat{\mathbf{p}} = \hat{z}\mathbf{r} = (sz + t)\mathbf{r} = (s + t/z)z\mathbf{r}$. □

Corollary 5.3 (Affine error in disparity). *If disparity is $\hat{d} = sd + t$ then $\hat{z} = 1/\hat{d}$ and $\hat{\mathbf{p}} = \mathbf{p}/(s + tz)$: a projective map of space.*

Proof. $\hat{z} = \frac{1}{s/z + t} = \frac{z}{s + tz}$, so $\hat{\mathbf{p}} = \hat{z}\mathbf{r} = \mathbf{p}/(s + tz)$. □

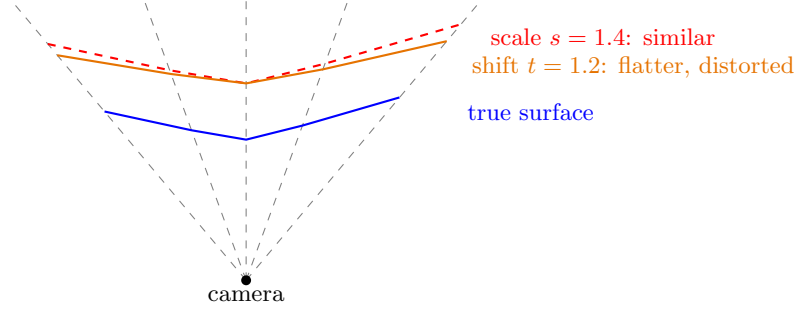


Figure 13: Original side view. A bowl-shaped surface seen along five rays. Scaling by s keeps the shape (red); adding a constant depth t along each ray (orange) keeps the depth differences but widens the lateral spread, so the bowl becomes flatter.

Intuition

Scale is a photocopier enlargement: bigger, same shape. Shift is pushing every point straight away from the camera by the same distance: because rays fan out, far-off directions move sideways more than the centre, so straight things bend. Example: $s = 1$, $t = 1$ m: a point at $z = 2$ is scaled by $1 + 1/2 = 1.5$, one at $z = 10$ by only 1.1; near points are magnified more, so a wall receding from the camera is bent.

5.2 Sources of Scale and Shift (*slides 34*)

Two numbers are required, and a single image cannot supply them. Possible sources:

- a **metric-depth model**; camera-aware models narrow the generalization gap by taking the intrinsics as input (3.8);
- a **known baseline** at training time (3.9);
- a **known object size**: the apparent-size cue, which needs recognition;
- **known camera height plus a flat-ground assumption**: the vertical-position cue, which needs flat ground;
- or a **handful of real depth measurements**.

6 Marigold-DC (*slides 35–38*)

Section 3.13: guidance from sparse depth measurements. Nothing is retrained: the denoising loop is modified at test time to move the sample toward the available measurements.

6.1 The Depth Completion Task (*slides 36*)

Definition 6.1 (Depth completion). Depth completion upgrades sparse depth measurements into dense depth maps guided by a conventional image. The sparse points come from the LiDAR returns of Part I, structure-from-motion landmarks, or a phone’s depth sensor.

[PHOTO PLACEHOLDER]

Insert the image from **slide 36**:

the comparison grid: ground truth, input image and the sparse guidance at high / medium / low density, with the outputs of CompletionFormer, BP-Net, VPP4DC and Marigold-DC and their RMSE labels

guidance density	CompletionFormer	BP-Net	VPP4DC	Marigold-DC
high	1.8	1.7	1.6	1.5
medium	3.7	2.9	2.0	1.6
low	4.8	3.2	3.3	2.0

RMSE values printed on slide 36 (units not stated on the slide).

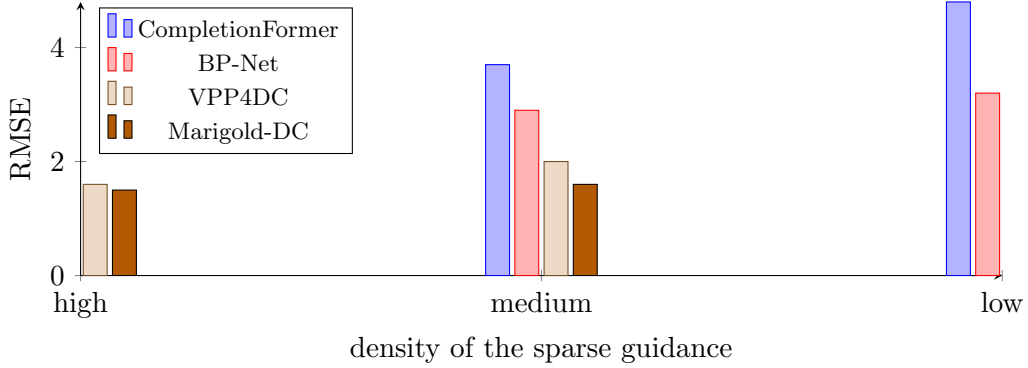


Figure 14: Plot of the RMSE numbers on slide 36. Baselines degrade quickly as the guidance gets sparser; Marigold-DC degrades slowly because the prior fills what the sensor did not sample.

6.2 Steering the Loop (*slides 37*)

Depth completion is reframed as *image-conditional depth generation guided by sparse measurements*. It is **generation rather than regression**: the output comes from a prior. The image **conditions** the network, the sparse depth **guides** the loop, and the measurements were never part of training. Every step already walks toward the model’s current estimate; a second pull is added, toward the measurements.

Algorithm 6.2 (Guided step). At every step of the loop: (1) read the *preview*, the current guess at the finished depth map, $\hat{z}_0^t = z_t - \sigma_t \epsilon_\theta$ (Tweedie formula); (2) decode it and turn it into meters with the scale and the shift, $\hat{d}_t^{(m)} = \hat{a} \mathcal{D}(\hat{z}_0^t) + \hat{b}$; (3) compare it with the few pixels Ω that have a measurement $\mathbf{c}$, $\mathcal{L} = \|\hat{d}_t^{(m)} - \mathbf{c}\|_\Omega$; (4) nudge the latent and the two scalars $(\hat{a}, \hat{b})$ toward the measurements by gradient descent on $\mathcal{L}$; (5) take the ordinary DDIM step.

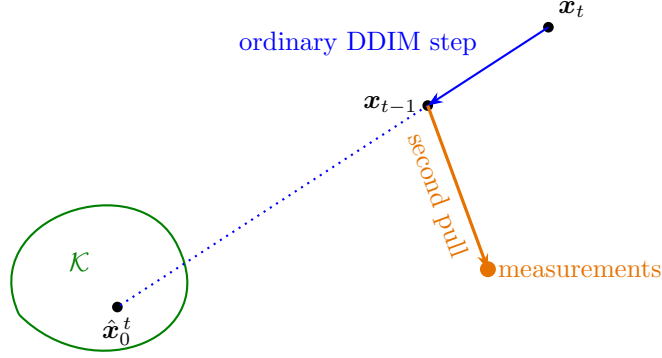


Figure 15: The guided step (redrawn from slide 37): DDIM step plus a second pull toward the sparse measurements.

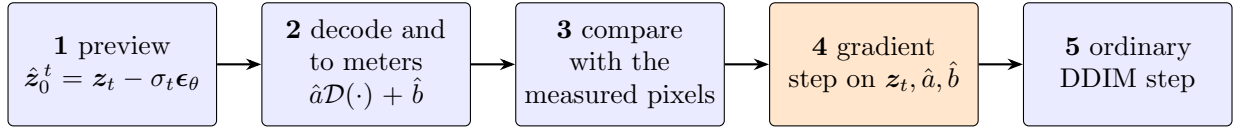


Figure 16: The five steps as a pipeline.

6.3 Nothing Retrained (*slides 38*)

Marigold gives affine-invariant depth; the sparse map gives *metric* depth at a few pixels. The unknowns are the **latent**, which decides the shape, and the **scale and shift**, which decide the meters. All three are optimized, for one image, over fifty steps, and then discarded. Gradients are taken with respect to the latent and the two scalars, so **no weights change**: the U-Net and the VAE are frozen throughout.

Theorem 6.3 (Zero-shot property, as claimed on the slide). *There is no depth-completion dataset, no architecture change and no new checkpoint. The method is therefore zero-shot across indoor and outdoor scenes, where the baselines ship one checkpoint per domain.*

Intuition

The frozen network is a very good but unit-less sketch artist; the LiDAR points are a few tape-measure readings. At each step the sketch is bent slightly (through the latent) so that it does not contradict the readings, and the ruler (scale and shift) is adjusted until the sketch reads in meters. Because the readings are used only at test time, the same model handles a room and a street: the debts of Section 1 are paid at once, the prior supplying what the sensor cannot sample and the sensor the two numbers the prior cannot know.

[PHOTO PLACEHOLDER]

Insert the image from **slide 38**:

the Marigold-DC diagram: goose photograph, encoder/decoder, frozen U-Net loop with the red gradient arrows, sparse map $\mathbf{c}$ and the resulting metric depth (the block structure is redrawn in the figures above)

7 Summary of Lecture 03 (*slides 39–40*)

Every method in this lecture bought depth with a different resource. The last section bought it with two at once.

method	paid with	when	free parameters left
A laser (3.3, 3.4)	time	test	0, but sparse
Stereo	a known baseline	test	0, error grows as z^2
Structure from motion	viewpoints, no baseline	test	1, one global scale
Supervised (3.7)	a prior, on sensor labels	train	0, 1 or 2, set by the labels
Self-supervision (3.9)	a baseline	train	0, for that rig
Marigold (3.11)	a generative prior	train	2
Marigold-DC (3.13)	a prior and time	both	0, dense, nothing retrained

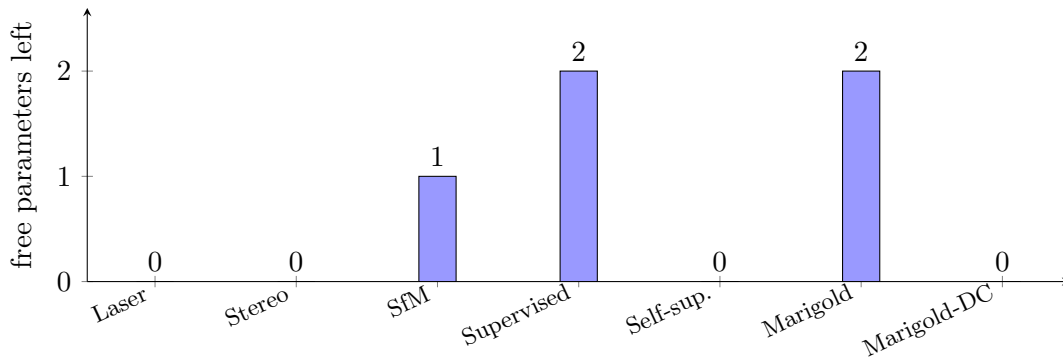


Figure 17: Free parameters left after each method (last column of the table). “Supervised” is drawn at its worst case: it ranges from 0 to 2 depending on the labels.

Key take-aways

1. **The prior supplies what the sensor cannot sample, and the sensor supplies the two numbers the prior cannot know.**
2. Self-supervision from stereo needs no depth labels, but the photometric loss is blind on flat and occluded pixels (hand-written priors fill the gap) and is metric only for the baseline it was trained with.
3. A diffusion denoiser is a prior: it learns the gradient of a smoothed distance to the data, is defined everywhere, and can be sampled, conditioned, or guided; regression by contrast returns the conditional mean, which blurs depth edges.
4. Marigold reuses Stable Diffusion’s weights: frozen VAE, U-Net fine-tuned with the image latent concatenated; percentile normalization makes the output affine-invariant, so it gives shape but no meters, no common zero, and costs $50 \times 10 = 500$ evaluations.
5. An affine error $\hat{z} = sz + t$ maps to $\hat{\mathbf{p}} = (s + t/z)\mathbf{p}$: scale is harmless to shape, shift distorts it. Marigold-DC fixes both by optimizing the latent, scale and shift at test time against a few metric points, with no retraining.

Acknowledgments and sources (*slides 41–42*)

The diffusion section (3.10) is adapted from *Introduction to Diffusion*, MIT 6.S183, IAP 2026, Lecture 1, by D. Pfommer; parts of both halves are adapted from *Depth Estimation* (Politecnico di Milano, 2024) by L. Magri. The reference list on slide 42 (Pfommer, Yuan, Ke et al., Rombach et al., Yang et al., Ranftl et al., Godard et al., Ho–Jain–Abbeel, Song–Meng–Ermon, Karras et al., and others) and further reading (Yuan’s smallldiffusion) are given there.

A Supplementary derivations (not on the slides)

The following results are *not* on the slides; they are added to justify remarks made in the main text.

Proposition A.1 (DDIM step = re-noising the clean estimate). *Equation (??) is equivalent to $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_0^t + \sigma_{t-1} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, \sigma_t)$.*

Proof. $\hat{\mathbf{x}}_0^t = \mathbf{x}_t - \sigma_t \boldsymbol{\epsilon}_\theta$, hence $\mathbf{x}_t = \hat{\mathbf{x}}_0^t + \sigma_t \boldsymbol{\epsilon}_\theta$ and $\mathbf{x}_{t-1} = \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \boldsymbol{\epsilon}_\theta = \hat{\mathbf{x}}_0^t + \sigma_{t-1} \boldsymbol{\epsilon}_\theta$. $\square$

Reading: keep the current clean estimate and re-attach the *same* predicted noise, only with the smaller amplitude σ_{t-1} .

Proposition A.2 (Irreducible floor of the training loss). *For the ideal denoiser, $L_{\text{simple}}(\boldsymbol{\epsilon}^*) = \mathbb{E}_{\mathbf{x}_\sigma, \sigma} \text{tr Cov}(\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma)$, which is strictly positive whenever the posterior of $\mathbf{x}_0$ given $\mathbf{x}_\sigma$ is not a point mass.*

Proof. Applying the conditional-mean decomposition of Section 3.1 with $Y = \boldsymbol{\epsilon}$ and $f = \mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma]$ gives $\mathbb{E} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}^*\|^2 = \mathbb{E} \text{tr Cov}(\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma)$; and $\boldsymbol{\epsilon} = (\mathbf{x}_\sigma - \mathbf{x}_0)/\sigma$, so $\text{Cov}(\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma) = \text{Cov}(\mathbf{x}_0 \mid \mathbf{x}_\sigma)/\sigma^2$. $\square$

Reading: this explains why the slide-14 curve stops around 0.41 rather than 0: the remaining loss is genuine ambiguity about which data point the noisy input came from, not an optimization failure.